

 **DTU Compute**
Department of Applied Mathematics and Computer Science

Artificial Intelligence and Human Cognition Lecture Notes

v. 1.25.2

©Tobias Andersen

These lecture notes are intended for use in teaching
at the Technical University of Denmark. Please do
not distribute or share outside their intended use.



Contents

Contents	i
3 Neural encoding	1
3.1 Linear encoding	1
3.2 Lateral inhibition	2
3.3 Principal Component Analysis (PCA) in linear encoding	7
Bibliography	11

CHAPTER 3

Neural encoding

3.1 Linear encoding

Neurons are the main information processors in the brain. They come different forms and the details of their functioning are, unsurprisingly, complex. We can, however, come to a basic understanding of how neurons encode information from a very simplified linear model.

Neurons generally have dendrites that derive their name from their tree-like appearance. The dendrites are dotted with receptors that can be activated by specific neurotransmitters sent from the axons, or nerve fibres, of other neurons. The dendrites thus receive input from other neurons while the axons send an output to other neurons.

The dendrites of a single neuron can receive input from many (thousands) of other neurons. All these input do not affect the neuron in the same way. Some affect the neuron more strongly than others. The effect can furthermore be either excitatory, which means that the input increase the activation level of the receiving neuron while others are inhibitory and have the opposite effect. The input to a neuron is thus *weighted* meaning that it is multiplied by a real scalar. If a neuron receives input from N neurons, a simple model for the activation level, a_l , is the linear expression

$$a_l = \mathbf{j}^T \mathbf{w}_l \quad (3.1)$$

where $\mathbf{j}$ is an N -dimensional vector containing the input the neuron receives and $\mathbf{w}_l$ is a N -dimensional vector containing the weights of the input to the neuron.

Extending to L neurons each receiving the same input, $\mathbf{j}$, the linear model can be written as

$$\mathbf{a}^T = \begin{bmatrix} a_1 \\ \vdots \\ a_L \end{bmatrix}^T = \mathbf{j}^T [\mathbf{w}_1 \quad \cdots \quad \mathbf{w}_L] = \mathbf{j}^T \mathbf{W} \quad (3.2)$$

Here, we will work with the simple linear model of neural encoding given by Equation 3.1. This model is an over-simplification in comparison to both biological neurons and the artificial neurons in neural networks that are used in machine learning. Note, that unlike a networks of these more complex, non-linear neurons, a network of linear neurons does not gain complexity from having multiple layers. To see why this is so, consider a two-layer linear network where the output of the first layer is $\mathbf{j}\mathbf{W}_1$. This output serves as the input to the second layer through weight matrix $\mathbf{W}_2$. We can then write the activation of the second layer as

$$\mathbf{a} = \mathbf{j}\mathbf{W}_1\mathbf{W}_2 = \mathbf{j}\mathbf{W}_{12}$$

Hence, a single layer model with weight matrix $\mathbf{W}_{12} = \mathbf{W}_1\mathbf{W}_2$ is mathematically equivalent to the two-layer model.

Biological neurons become active and send an action potential down their axon only when they receive activation above a certain threshold. The action potential causes the release of neurotransmitters that activate other neurons. The action potential is binary, often referred to as all-or-nothing in the biological literature. However, this does not mean that neurons are binary like a simple on-off switch or a bit because the *rate* at which action potentials occur is very important for how they activate other neurons. If a neuron is activated far above its threshold, then it will fire at a high rate, activating

other neurons more strongly than if it were activated only slightly above its threshold. We can model this using an activation function, but first we should clarify a somewhat confusing terminology: In the neurobiological literature, the activation level of the neuron describes how much it is activated. This is the activation described by Equation 3.1. This activation level is the input to the activation function. The output of the activation function is the degree to which the neuron activates other neurons. Here, we will not provide an exact activation function for biological neurons, but only note that the output of the function is zero (or has a constant baseline value) when the activation level is below the threshold, it increases with increasing activation above the threshold, and it is limited by a maximum since biological neurons will naturally have a maximum firing rate.

Neurons in artificial neural networks also employ an activation function. In the neural network literature, activation typically refers to the *output* of the activation function, which is, confusingly, not the same usage of the term as is found in the neurobiological literature. Commonly used activation functions include sigmoid (s-shaped) activation functions such as the hyperbolic tangent function and the logistic function. These functions are monotonically increasing and have an upper bound, but they are different from the activation function of biological neurons in that they do not have well-defined threshold under which the function takes on the value of zero (or a constant baseline level). The rectified linear unit (ReLU) activation function is also commonly used. It does have a well-defined threshold and, above this threshold, it is monotonically increasing, but it is different from the activation function of biological neurons in that it has no upper bound.

Biological neurons and neurons in artificial neural networks both rely on non-linear activation functions. This non-linearity makes them more flexible, allowing them to model both linear and non-linear input-output relationships. This flexibility is key to their effectiveness in information processing. However, this same flexibility has also contributed to the challenge of understanding how artificial neural networks process information and how to use them effectively. While non-linear neural networks gained significant attention in the 1980s, they only gained wider success the 2010s.

Despite the importance of non-linear activation functions for information processing in both biological and artificial neural networks, we focus exclusively on linear encoding here. One reason for this is that it is often difficult to qualitatively understand how non-linear neurons process information. In the machine learning literature this is known as the problem of *explainability*. Another reason is that although non-linear information processing is an important aspect of how the brain works, it is still possible to develop an understanding of information processing in the brain using linear models, as we shall see in the following sections.

3.2 Lateral inhibition

The first stage of encoding visual information happens in the photoreceptors in the retina of the eye. These cells are specialised *sensory* neurons that do not have dendrites that allow them to be activated by other neurons. Instead they have similar structures that allow the cells to be activated by light. There are two types of photoreceptors: rods and cones. Rods are more sensitive to light than cones. Therefore they are more important for our vision at low light. Cones are less sensitive to light. There are three types of cones. Each of these three types are more sensitive to light at a particular part of the visual spectrum. The combination of information from each type of cone is the basis of colour vision. In this section, we shall not consider colour vision, for simplicity. The photoreceptors are thus comparable to pixels in gray-scale digital images and their activation levels represent the brightness level of the a small area of the retina.

The second stage of encoding occurs also in the retina before information transmitted to the brain. In the retina, the axons of the photoreceptors project onto bipolar cells. A bipolar cell will be activated by a photoreceptor but, at the same time, it will be inhibited by the neighbouring photoreceptors. This simple network structure is called *lateral inhibition* and is repeated across the retina.

Lateral inhibition gives the bipolar cells an interesting property: they encode visual contrast rather than brightness. To see why this is so imagine that light shines on all the photoreceptors that project onto a bipolar cell. The activation level of the bipolar cell will increase due the excitation it receives from

one photoreceptor but, at the same time, it will be inhibited by the other neighbouring photoreceptors. If we set the weight of the excitatory cell to 1 and the weight of the inhibitory cells to w then we can write the one-dimensional case as

$$a_m = wj(m-1) + j(m) + wj(m+1) = \mathbf{w}^T \mathbf{j} \quad (3.3)$$

where $\mathbf{w}^T = [w \ 1 \ w]$ and $\mathbf{j}^T = [j(m-1) \ j(m) \ j(m+1)]$.

Note that, the last term in Equation 3.3 is the same as in Equation 3.1. Lateral inhibition is thus a form of linear encoding. Note also that the input vector, $\mathbf{j}$ must be two elements longer than the activation vector, here a scalar a_m . This is because we do not have a model for how to calculate the activation of the cells that receive input from the edge of the image. Therefore, we will ignore those cells in the following. Also note that if we set the weight of the inhibitory cells to $w = -0.5$ then the activation will be zero, $a_m = 0$ if the input is constant, $j(m-1) = j(m) = j(m+1)$. Hence, the neuron will not respond to an isoluminant input, but only respond to changing input, i.e. contrast. The bipolar cells thus encode local contrast and the information processing this simple network can thus be thought of as contrast enhancement.

We can see the function of the lateral inhibition network in another way if we rephrase Equation 3.3 as

$$a_m = \sum_{n=-1}^1 j(m+n)w(n) \quad (3.4)$$

Equation 3.4 describes lateral inhibition as a *convolution* of the input with a kernel, $\mathbf{k}_1^T = \mathbf{w}^T = [w \ 1 \ w]$. The network can thus be thought of as a spatial filter where the kernel has been chosen to enhance contrast.

Perceptual illusions are often a striking way to demonstrate the effects of neural information processing and the effects of lateral inhibition is no exception. Consider the Mach band illusion in the top panel of Figure 3.1. At the location of the rightmost arrow you can see a bright vertical band. At the location of the leftmost arrow you might also be able to see a dark band although it is less obvious than the brighter band. You might better see the illusory bands if you change the size of the figure or your viewing distance. The illusion might also be clearer if you do not look directly at the bands. Both bands are illusory. The horizontal luminance profile of the image is shown in the middle panel of the figure: it is a linear transition between the darker area in the left of the image to the brighter area in the right of the image. The vertical luminance profile is constant.

Why do we see the illusory Mach bands? Lateral inhibition may be part of the answer. The activation of a horizontal layer of neurons, that receive lateral inhibition, is plotted in the lower part of Figure 3.1. Notice that the activation is lowest near the location of the darker Mach band and highest near the location of the brighter Mach Band. This is because the image *changes* more at the locations. It is these changes, i.e. contrast, that our visual system tends to exaggerate through lateral inhibition.

Note that when we analyse images such as the Mach band image in the top panel of Figure 3.1 in terms of lateral inhibition, we might not get the expected results if we expect that each pixel activates only one single photoreceptor. If, for instance, each photoreceptor is activated by two pixels in the image then we should choose our kernel to be $\mathbf{k}_2 = [w \ w \ 1 \ 1 \ w \ w]$. If each photoreceptor is activated by three pixels in the image then we should choose our kernel to be $\mathbf{k}_3 = [w \ w \ w \ 1 \ 1 \ 1 \ w \ w \ w]$ and so forth. We will say that these filters has a *scale* of two and three, respectively. For each image, We will have to adjust this scale of the filter in order to get the expected activation levels. The activation profile in the the bottom panel of Figure 3.1 was, for example, created by filtering the image in the top panel with kernels of varying scales and then selecting the scale that gave an activation profile that approximates the perceived illusion.

Images and the retinal array of photoreceptors are, of course, two-dimensional. In two dimensions, we can still use convolution and spatial filtering as a model of lateral inhibition but we must use a matrix kernel. The kernels at scale 1 and 2 are

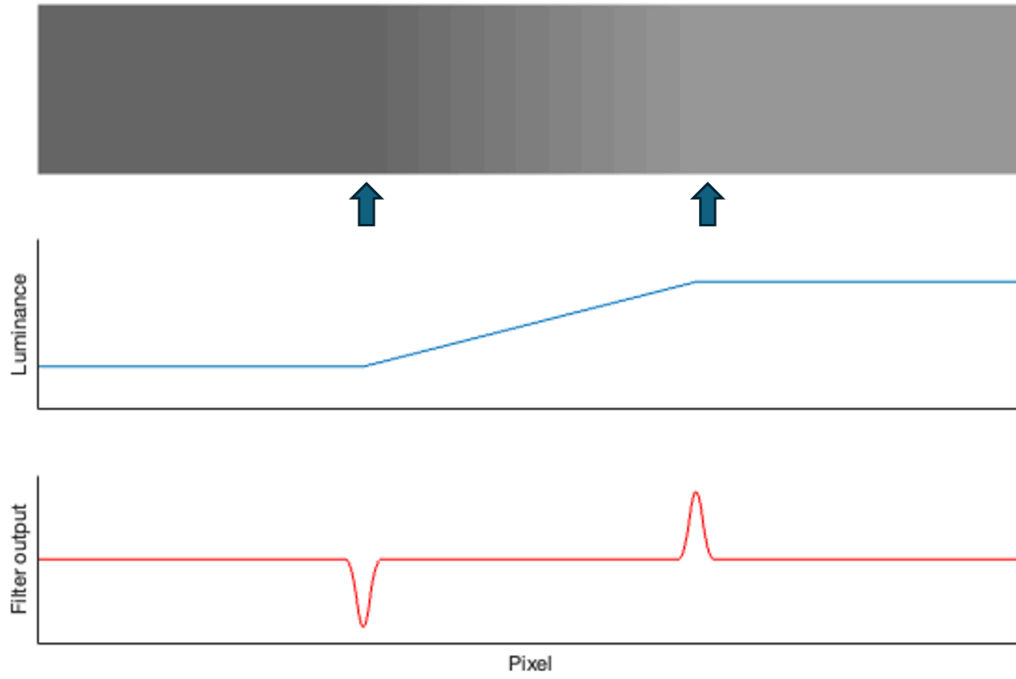


Figure 3.1: In the Mach band illusion in the top panel we perceive a dark and a bright band at the location of the two arrows. The luminance profile plotted in the middle panel does, however not change at the location of the illusory bands. It is a simple linear gradient between a dark and a lighter intensity. The activation profile of a layer of cells that receive lateral inhibition, plotted in the middle panel, is lowest at the location of the dark Mach band and highest at the location of the bright Mach band. This shows that lateral inhibition can explain, in part, the illusion..

$$\mathbf{K}_1 = \begin{bmatrix} w & w & w \\ w & 1 & w \\ w & w & w \end{bmatrix} ; \quad \mathbf{K}_2 = \begin{bmatrix} w & w & w & w & w & w \\ w & w & w & w & w & w \\ w & w & 1 & 1 & w & w \\ w & w & 1 & 1 & w & w \\ w & w & w & w & w & w \\ w & w & w & w & w & w \end{bmatrix} \quad (3.5)$$

Note that if we still want the filter to give an output of zero when the input is constant in the two-dimensional case, the weight the inhibition should be adjusted to $w = -0.125$. Note also that convolution in two dimensions will give the same edge effects as convolution in one dimension: At a scale of 1, the image must be 3-by-3 pixels and at scale of 2, the image must be 6-by-6 pixels. Finally, note that the kernel matrix, $\mathbf{K}$ is *not* equivalent to the weight matrix, $\mathbf{W}$, in Equation 3.2. If we want to write two-dimensional convolution in terms of the linear model, we must first flatten the image and kernel matrices, $\mathbf{J}$ and $\mathbf{W}$. Only then can we use Equation 3.2 to describe convolution in two dimensions. Although the two approaches are mathematically equivalent, it is more practical to use software convolution functions to calculate the activations across the entire image.

The left image in Figure 3.2 shows a photograph of da Vinci's original painting of Mona Lisa. The center image shows the original image after being filtered with a kernel with appropriate scale. Notice how the areas with high contrast have been enhanced. This is seen in particularly at the border between Mona Lisa's dark hair and the brighter sky behind. It is also seen at the border between Mona Lisa's

fair hands and her darker dress. Notice also the edge effects at the edges of the image. These arise because the entire original image was used to calculate the activations in the center image.

As a final stage of processing, we can move beyond the linear model and apply a threshold, so that all activations above a certain threshold are given a value of 1 and while all the other activations assume a value of 0. The result, shown in the right image in Figure 3.2, illustrates that the effect of thresholding is similar to *edge detection*. The image now resembles a line drawing with only the most prominent lines visible.



Figure 3.2: Left: A photograph of da Vinci's original painting of Mona Lisa. Center: When filtered using a kernel of appropriate scale the contrast is enhanced. This is seen in particularly at the border between Mona Lisa's dark hair and the brighter sky behind. It is also seen at the border between Mona Lisa's fair hands and her darker dress. Right: When filtered image is thresholded it resembles a line drawing showing that lateral inhibition followed by thresholding works as *edge detection*.

Simple cells in the visual cortex lie further up in the processing pathway of the visual system. These cells can have more complex response patterns. Some simple cells respond particularly strongly to edges with a specific orientation. This type of behavior can be modelled by a simple network structure where a number of bipolar cells that lines up in the retina excites a simple cell. The example in Figure 3.2 illustrates this.

We will end this section on lateral inhibition by examining another visual illusion, the Hermann grid illustrated in the right panel of Figure 3.3. When looking at the grid you can notice light gray shadows at the intersection of the white lines. Strangely, the shadows are only apparent when you do not look directly at them. This is the Hermann grid illusion. Lateral inhibition can, in part, explain the illusion: At the intersection of the white bars, the bipolar cells receive inhibition from the top, bottom, left and right because these areas are white. In comparison, in a vertical bar away from intersections, the bipolar cells receive inhibition only from the bottom and the bottom. That additional inhibition that the bipolar cells receive at the intersections is what makes them appear less bright than the rest of the bar, hence the gray shadows. The effect can be mimicked by convoluting the original image with a kernel of appropriate size. The convoluted image, will indeed have show the gray shadows at the intersection of the wite lines. This is shown in the left panel of Figure 3.3. So, why do the gray shadows disappear when you look directly at them. The reason may lie in that the density of photoreceptors is much higher in the *fovea* of the retina, which is at the focus of our gaze. In the periphery of the retina, the density of photoreceptors is much less. Hence, the spatial resolution of peripheral vision is much lower, which is why it is very difficult to read text if you do not look directly at it. In terms of lateral inhibition, this means that the kernel size is larger for peripheral vision because more there are fewer photoreceptors per pixel, which means that the kernel size should be larger. Conversely, for foveal

vision, the kernel size should be smaller. In agreement with this, the gray shadows do indeed vanish if the kernel size is made smaller than the one used in the right panel of Figure 3.3.

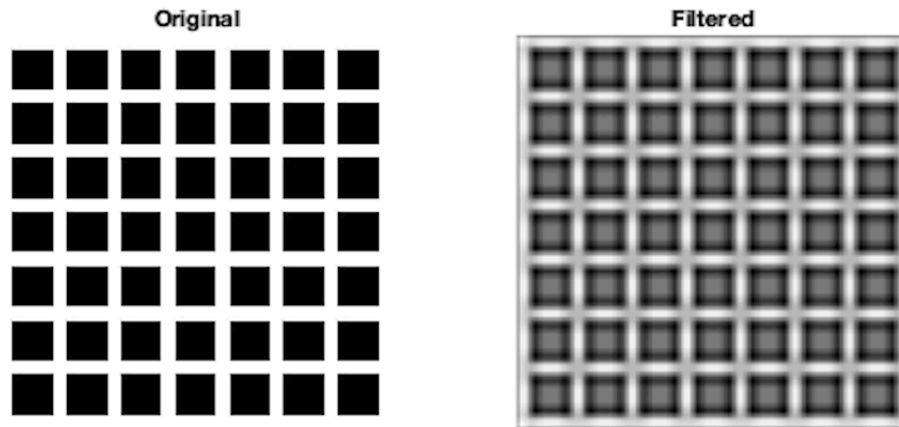


Figure 3.3: Left: The Hermann grid illusion. Gray shades seem to appear at the intersection of the white lines when you do not look directly at them. Strangely, they seem to vanish when you look directly at them. Right: The original image filtered by a kernel with appropriate scale. Notice how the filter has created gray shades at the intersection of the white lines. The activation thus mimicks the Hermann grid illusion.

3.2.1 Exercise - Lateral inhibition

The purpose of this exercise is to show that it is possible to construct simple quantitative models of neural systems and that these models can inform us about brain function and perception. In this exercise, we will examine a simplified model of lateral inhibition in the retina.

1. Assume that we have a 1-dimensional array of M cells. Each cell has an activation level a_m where $m = 2, \dots, M+1$. Each cell receives input j_n from photoreceptor stimulated by light. In addition, each cell receives scaled inhibitory input from neighboring photoreceptors so that

$$a_m = j_m + w(j_{m-1} + j_{m+1}) \quad \text{for } m = 2, \dots, M+1 \quad (3.6)$$

Calculate the activations when input is a 1-dimensional edge, i.e. a steep transition from dark to light. You can use an input vector $\mathbf{j}^T = [0 \ 0 \ 0 \ 0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1]$ and a weight, $w = -0.5$. Plot the values and explain how you can see that the network enhances contrast.

2. We can express Equation 3.6 as

$$a_m = \sum_{n=-1}^{n=1} j(m+n)k(n) \quad (3.7)$$

where k is a discrete kernel function with the elements $k(-1) = k(1) = -w$ and $k(0)=1$. Equation 3.7 is the formula of a 1-dimensional convolution showing that the function of the network is to convolve the input with the kernel, k . Recalculate the activation levels using one-dimensional convolution. There are functions for this in most software for scientific computing. Show that you get the same activation values as you got before.

3. Neurons are only active when the sum of the activation they receive from other neurons exceed a certain threshold. Use a threshold function on the activations so that the network works as an edge-detector in one dimension. Which value do you set the threshold to? Explain your choice.
4. Recreate Figure 3.1. Explain your approach. Describe how you created the luminance profile in the middle panel of Figure 3.1. Which kernel scale did you use?
5. In two dimensions, the neurons would receive inhibitory input from neighboring photoreceptors to the left and right and above and below. This would turn the kernel function into a matrix, $\mathbf{K}$, as described in Equation 3.5. Load the image of Mona Lisa that comes with this exercise and use 2-dimensional convolution to recreate Figure 3.2. Decide which kernel size works best for contrast enhancement and which threshold values works best for edge detection. Explain your approach.
6. Load the image of the Hermann grid that comes with the exercise and use 2-dimensional convolution to recreate Figure 3.3. Decide which kernel size works best for recreating the figure. Explain your approach.

3.3 Principal Component Analysis (PCA) in linear encoding

In Equation 3.2 we described linear encoding of N inputs into L neurons through a matrix, $\mathbf{W}$, of weights. In this section, we will use PCA as a model for how the number of neurons, L , and the values of their weights is determined.

Before we delve into the details of PCA we will consider how to construct the network in more general terms. First, we will notice that it will be good to minimise the number of neurons, L , because neurons take up energy, limited by the food supply, and space, particularly limited within the protective enclosure of the skull. Second, it also seems reasonable that we will need as many neurons as we have inputs, $L = N$, to avoid loss of information. We may, however, accept some loss of information to limit the number of neurons required. In machine learning terms, this is the problem of reducing the dimensionality of the input with as little loss of information as possible.

To minimise the number of neurons, L , they should not encode redundant information. This means that their activation levels across many stimuli should be independent. If the distribution of the activations levels are Gaussian, this means that activation levels of the neurons should have zero correlation. Notably, many studies have shown that this is *not* the case for the inputs. One way to show this is to simply take a digital camera and record many images. The intensity values of neighbouring pixels will be correlated across the images. This simply reflects the structure of our environment that is made up of large surfaces, or homogeneous patches. Most often, the image such a surface or patch will cover multiple photoreceptors although, occasionally, an edge or some form of local contrast will cause neighboring photoreceptors to have very different activation levels. Based on these observations, it seems that we might be able to find a network structure with fewer neurons than inputs, $L < N$, while minimising the loss of information by getting rid of the correlation between the inputs. PCA can do exactly that.

We will not go through all the details of PCA as this is covered elsewhere, e.g. Chapter 3 in Herlau, Schmidt and Mørup Schmidt (Herlau et al., 2021). In stead we will focus on relating PCA to neural encoding.

PCA takes an M -by- N data matrix, in our case an matrix of images, $\mathbf{J}$, where each row is an observation, in our case an image, and each column correspond to a feature, in our case a pixel. The matrix must have zero mean.

PCA factorises this matrix into two matrices, $\mathbf{A}$ and $\mathbf{W}^T$, so that

$$\mathbf{J} = \mathbf{A}\mathbf{W}^T \quad (3.8)$$

In PCA terminology, the columns of $\mathbf{W}$, contains the *principal components*, or *loadings*. The principal components are vectors that form a new basis in the original data space. If the data matrix

contains at least as many observations as features, i.e. if $M \geq N$, then $\mathbf{W}$ is a square N -by- N matrix, so that the number of principal components is equal to the dimensionality of the original data space, which means that the principal components form a complete orthonormal basis. In terms of the neural model of linear encoding, this means that the number of neurons, L , is equal to the number of inputs, N . In that case, $\mathbf{A}$ is an M -by- N matrix in which each row $\mathbf{A}$ contains an image represented in this basis. Notably, the columns of $\mathbf{A}$ are orthogonal. In terms of the neural model of linear encoding, this means that the activation levels of the neurons are not correlated across the images. Since the principal basis is complete, this representation of the data is also complete meaning that we can reconstruct the data matrix $\mathbf{J}$ from the matrices $\mathbf{A}$ and $\mathbf{W}$ without any loss of data. Concordantly, matrices $\mathbf{A}$ and $\mathbf{J}$, are of the same size.

To better understand how PCA relates to linear encoding, we can isolate the matrix $\mathbf{A}$. Since $\mathbf{W}$ is orthonormal, $\mathbf{W}^T \mathbf{W} = \mathbf{I}$, we can isolate $\mathbf{A}$ by right multiplying Equation 3.8 by $\mathbf{W}$

$$\mathbf{A} = \mathbf{J}\mathbf{W} \quad (3.9)$$

If we extract a single image (a row) from the matrices $\mathbf{A}$ and $\mathbf{J}$ then we get the model of linear encoding in Equation 3.2 with one noticeable exception: In Equation 3.2, the weight matrix, $\mathbf{W}$, was N -by- L , where N is the dimensionality of the input (the number of pixels) and L is the number of neurons that receive this input. In Equation 3.9 the weight matrix is N -by- N meaning that the two models are only equivalent if the number of neurons that receive the input is equal to the dimensionality of the input, $L = N$.

PCA is often used for dimension-reduction in machine learning. The orthogonality of the columns of $\mathbf{A}$ makes it particularly suited for this. In statistical terms, the orthogonality means that the data in this representation has zero covariance. Informally, another way to think of this is that all variance that can possibly be contained in one column of $\mathbf{A}$ is indeed contained in that column. The objective of minimising the covariance between the columns is thus equivalent to maximising the variance of each column. Furthermore, PCA sorts the columns of $\mathbf{A}$ by the amount of variance they contain, so that the first column of $\mathbf{A}$ contains the most variance and the last column the least. Therefore, we can reduce the dimension of the neural representation, $\mathbf{A}$, from N to L , by selecting only the first L columns of $\mathbf{A}$ and the corresponding first L principal components (columns of $\mathbf{W}$) knowing that we have kept as much variance as possible in L dimensions.

The mathematical equivalence of the linear neural network in Equation 3.2 and Equation 3.9 allow us to interpret PCA in terms of neural encoding. Each principal components, i.e. each column of $\mathbf{W}$ correspond to a set of neural weights for one neuron. Each row in $\mathbf{A}$ represents the data, i.e. an image, as activations of the neurons.

There is an important difference between Equation 3.8 and Equation 3.9 that their mathematical equivalence may hide. As we have already noticed Equation 3.9 describes neural encoding. Knowing the input, $\mathbf{J}$, and the neural weights, $\mathbf{W}$, we can calculate the activation, $\mathbf{A}$ of the neurons. Equation 3.8 describes neural *decoding*. Knowing the activation of each neuron, $\mathbf{A}$, and the neural weights, $\mathbf{W}$, we can calculate the data, i.e. the image, from the neural representation. This is an important subsequent stage of neural processing. There is no point in only encoding the information; the brain must also be able to decode it so that it can act on it.

In the previous section we saw how a linear network with lateral inhibition performs a convolution, or spatial filtering, of images using kernels like the ones in Equation 3.5. We also briefly discussed how the combination of the output of this filter can be equivalent to detecting edges in images and that neurons in the visual cortex do indeed respond to edges of a particular location and orientation. How does the brain decide to use this particular type of filters? Whereas lateral inhibition in the retina may be *hard wired*, meaning that the development of this network structure and corresponding kernels is genetically pre-determined, we know that the network structure and corresponding kernels in the cortex are determined during development and requires varied visual input. One famous study demonstrated this by raising kittens in an environment with only horizontal lines (Blakemore and Cooper, 1970). When the kittens were moved to a more visually varied environment, they were poor at discriminating

vertical lines. Thus it seems that the visual system learns which kernels to use and PCA serves as a simple model for how it does that.

3.3.1 PCA of image patches

As an example, we will return to the image of Mona Lisa in Figure 3.2 and use PCA to learn kernels at a scale of 10 that will allow us to encode most of the variance in the image. First, we will sample 10-by-10 image patches from the image. We can do this at random or we can split the image into non-overlapping patches. We will then flatten the 10-by-10 patches so that they become 100-dimensional vectors. For each pixel, we must also subtract the mean across images, so that the vectors will have a mean of zero across images. Then we can perform PCA, which will give us the matrices $\mathbf{A}$ and $\mathbf{W}$.

Recall that the principal components, the columns of $\mathbf{W}$, are vectors in the original data space, here of 100 dimensions corresponding to flattened 10-by-10 image patches. We can thus visualise the principal components as image patches by unflattening them. Figure 3.4 shows the first 6 principal components as image patches and allow us to interpret the components in the image that the principal components capture. The first principal component is an almost uniform patch. This is the component that captures the overall luminance of the patch. The second principal component has positive values at the top and negative values at the bottom. It will capture a horizontal edge dividing the patch. Similarly, the third principal component will capture vertical edges. The fourth principal component is particularly interesting because it resembles the kernels that came from the lateral inhibition network in Section 3.2: it represents contrast between the center and the periphery of the patch. Finally, note that the sixth patch has a more complex structure representing horizontal lines.

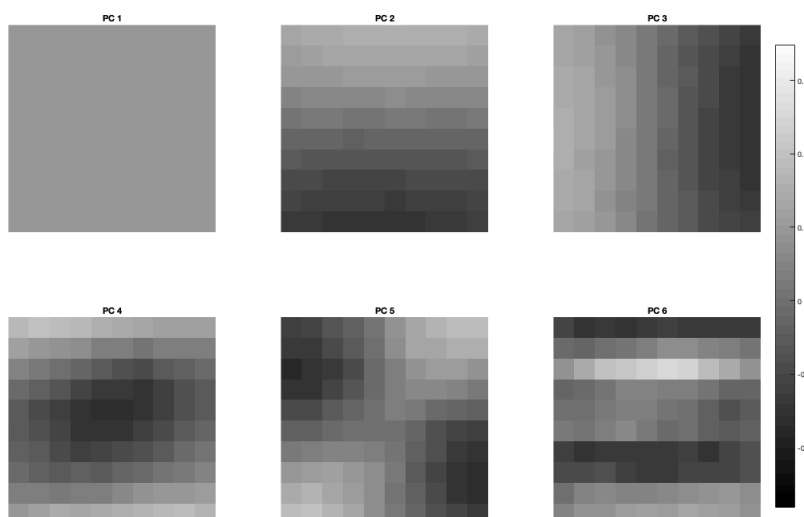


Figure 3.4: The first 6 principal components learned from 10-by-10 image patches taken from Mona Lisa. Note that the patches are shown on the same color scale shown in the color bar to the right..

We can see the effect of encoding the image patches using $L < N$ principal components by reconstructing the image patches using only L principal components. We do this using Equation 3.8 selecting only the first L columns of matrices $\mathbf{A}$ and $\mathbf{W}$. Note that we must also remember that before PCA we subtracted the mean of $\mathbf{J}$ from the rows of $\mathbf{J}$, so we will have to add it back to the reconstruction. Finally we have to unflatten the 100-dimensional vector so that they become 10-by-10 pixel image patches.

Figure 3.5 illustrates the reconstruction of a set of image patches that form a detail from the original image in the left panel of Figure 3.2. In the center panel, the patches are reconstructed using the first

principal components. Although the image is more blurry, the reconstruction captures a large part of the variance in the original shown in the left panel. This shows that PCA has managed to compress a large part of the variance in the image onto only six components. With a weight matrix determined by PCA, a linear neural network would thus be able to compress the dimension of the visual input substantially with a modest loss of information. Finally, note that in the right panel, the patches have been reconstructed using only the first principal component. This image is a pixelated version of the original. This is because all the image patches are represented by only one component, which is, as we note above, a uniform patch. The image patches are therefore all uniform varying only in intensity, which is what gives the image its pixelated appearance.



Figure 3.5: Left: A detail from the original image Mona Lisa in Figure 3.2. Center: The same detail made of image patches reconstructed using six principal components. Right: The same detail reconstructed using only one principal component. See main text for details..

3.3.2 Exercise - Learning image features using Principal Component Analysis

Load the image of Mona Lisa that you find in the exercise folder and transform it to a set of non-overlapping patches each 10-by-10 pixels. Flatten the patches into 100-dimensional vectors. Stack the vectors into a matrix, $\mathbf{J}$, where each row is an observation, in our case an image patch, and each column correspond to a feature, in our case a pixel. Use PCA to recreate Figures 3.4-3.5. You do not have to select the exact same detail of Mona Lisa that was selected in Figure 3.4. Repeat the process for other image patch sizes and other images. Compare your image reconstructions and your principal components across image patch sizes and other images to see if you get similar results.

Bibliography

- Blakemore, C., & Cooper, G. F. (1970). Development of the brain depends on the visual environment. *Nature*, 228(5270), 477–478. <https://doi.org/10.1038/228477a0>
- Herlau, T., Mørup, M., & Schmidt, M. N. (2021). *Introduction to machine learning and data mining*. DTU lecture notes. <https://gitlab.compute.dtu.dk/tuhe/books>

